
VisionPipe

Release 0.1.0

Nathaniel Yeo, Muhammad Tufail

May 27, 2026

CONTENTS:

1	Subpackages	1
1.1	src.data_intake package	1
1.1.1	Subpackages	1
1.1.1.1	src.data_intake.data_intake package	1
1.2	src.data_stream package	10
1.2.1	Submodules	10
1.2.1.1	src.data_stream.StreamConversionServer module	10
1.3	src.deep_stream package	11
1.3.1	Submodules	11
1.3.1.1	src.deep_stream.api module	11
1.3.1.2	src.deep_stream.deepstream module	11
1.4	src.model_deploy package	12
1.5	src.model_training package	12
1.5.1	Submodules	12
1.5.1.1	src.model_training.trainer module	12
1.6	src.pig_sorting_api package	16
1.6.1	Submodules	16
1.6.1.1	src.pig_sorting_api.client module	16
1.6.1.2	src.pig_sorting_api.main module	16
1.7	src.pig_sorting_streamlit package	32
1.7.1	Submodules	32
1.7.1.1	src.pig_sorting_streamlit.app module	32
	Python Module Index	35
	Index	37

SUBPACKAGES

1.1 src.data_intake package

1.1.1 Subpackages

1.1.1.1 src.data_intake.data_intake package

Submodules

src.data_intake.data_intake.Client module

class src.data_intake.data_intake.Client.**datetime**(year, month, day[, hour[, minute[, second[,
microsecond[, tzinfo]]]]])

Bases: *date*

The year, month and day arguments are required. tzinfo may be None, or an instance of a tzinfo subclass. The remaining arguments may be ints.

astimezone()

tz -> convert to local time in new timezone tz

classmethod combine()

date, time -> datetime with same date and time fields

ctime()

Return ctime() style string.

date()

Return date object with same year, month and day.

dst()

Return self.tzinfo.dst(self).

fold

classmethod fromisoformat()

string -> datetime from a string in most ISO 8601 formats

classmethod fromtimestamp()

timestamp[, tz] -> tz's local time from POSIX timestamp.

hour

isoformat()

[sep] -> string in ISO 8601 format, YYYY-MM-DDT[HH[:MM[:SS[.mmm[uuu]]]]][+HH:MM]. sep is used to separate the year from the time, and defaults to 'T'. The optional argument timespec specifies the number of additional terms of the time to include. Valid options are 'auto', 'hours', 'minutes', 'seconds', 'milliseconds' and 'microseconds'.

max = `datetime.datetime(9999, 12, 31, 23, 59, 59, 999999)`

microsecond

min = `datetime.datetime(1, 1, 1, 0, 0)`

minute**classmethod now(tz=None)**

Returns new datetime object representing current time local to tz.

tz

Timezone object.

If no tz is specified, uses local timezone.

replace()

Return datetime with new specified fields.

resolution = `datetime.timedelta(microseconds=1)`

second**classmethod strptime()**

string, format -> new datetime parsed from a string (like time.strptime()).

time()

Return time object with same time but with tzinfo=None.

timestamp()

Return POSIX timestamp as float.

timetuple()

Return time tuple, compatible with time.localtime().

timetz()

Return time object with same time and tzinfo.

tzinfo**tzname()**

Return self.tzinfo.tzname(self).

classmethod utcfromtimestamp()

Construct a naive UTC datetime from a POSIX timestamp.

classmethod utcnow()

Return a new datetime representing UTC day and time.

utcoffset()

Return self.tzinfo.utcoffset(self).

utctimetuple()

Return UTC time tuple, compatible with time.localtime().

src.data_intake.data_intake.NVRConnection module

class src.data_intake.data_intake.NVRConnection.**CapturePayload**(*args: Any, **kwargs: Any)

Bases: BaseModel

Payload model for manual image capture requests.

channels

List of camera channels to trigger captures/pictures on.

Type

List[int]

comment

This will be able to hold a comment for all cameras in channels.

Type

str, optional

comments

comments for individual cameras mapped by camera channels.

Type

dict[int, str], optional

channels: List[int]

comment: str | None = None

comments: dict[int, str] | None = None

class src.data_intake.data_intake.NVRConnection.**ChannelsRequest**(*args: Any, **kwargs: Any)

Bases: BaseModel

Request model for retrieving active and connected camera ids.

username

The username to log into the NVR.

Type

str

password

The password to log into the NVR.

Type

str

nvr_ip

The ip to log into the NVR.

Type

str

nvr_ip: str | None = None

password: str | None = None

username: str | None = None

```
class src.data_intake.data_intake.NVRConnection.FrameIntervalRequest(*args: Any, **kwargs: Any)
```

Bases: BaseModel

Request model for updating camera frame intervals.

channel

The target camera which is tied to port number.

Type

int

interval

Number of frames to skip between captures for this channel.

Type

int

channel: int

interval: int

```
class src.data_intake.data_intake.NVRConnection.ImageMetadata(*args: Any, **kwargs: Any)
```

Bases: BaseModel

Model representing metadata for a captured image.

image_path

Path to the saved image file.

Type

str

author

Name or identifier of the user/system that captured the image.

Type

str

serial_number

Serial number of the camera.

Type

str, optional

date_time

Timestamp of when the image was captured.

Type

datetime.datetime

user_comment

User-provided comment or note.

Type

str, optional

description

Additional description of the image.

Type

str, optional


```

class Config
    Bases: object
    arbitrary_types_allowed = True

    author: str
    date_time: datetime
    description: str = None
    image_path: str
    serial_number: str = None
    user_comment: str = None

```

```
src.data_intake.data_intake.NVRConnection.Lock()
```

allocate_lock() -> lock object (allocate() is an obsolete synonym)

Create a new lock object. See help(type(threading.Lock())) for information about locks.

```
exception src.data_intake.data_intake.NVRConnection.RequestException(*args, **kwargs)
```

Bases: OSError

There was an ambiguous exception that occurred while handling your request.

```
src.data_intake.data_intake.NVRConnection.SavePictureData(image_path, author, serialNumber,
                                                           dateTime, userComment, description,
                                                           trigger_method)
```

Write EXIF metadata to an image and store metadata in MongoDB.

Parameters

- **image_path** (*str*) – Path to the saved image.
- **author** (*str*) – Name or identifier of the capture source.
- **serialNumber** (*str*) – Device serial number.
- **dateTime** (*datetime.datetime*) – Timestamp of capture.
- **userComment** (*str*) – User-provided comment.
- **description** (*str*) – Description of the image content.
- **trigger_method** (*str*) – How the capture was triggered (e.g., manual, interval).

Behavior:

- Writes EXIF metadata into the image file.
- Inserts metadata document into MongoDB.

Todo

add trigger_method to the database

```
src.data_intake.data_intake.NVRConnection.capture_camera(username, password, nvr_ip, channel,  
                                                         subtype=0)
```

Continuously capture frames from a camera channel, saving images and video.

Parameters

- **username** (*str*) – NVR username.
- **password** (*str*) – NVR password.
- **nvr_ip** (*str*) – IP address of the NVR.
- **channel** (*int*) – Camera channel to capture from.
- **subtype** (*int, optional*) – Stream subtype (e.g., 0 = main stream, 1 = substream). Defaults to 0.

Behavior:

- Saves continuous video stream to *.avi* file.
- Saves periodic still frames based on channel frame interval.
- Supports manual capture triggers via *manual_capture_flags*.
- Stops gracefully when *stop_event* is set.

Logs:

- Frame dimensions, FPS, capture status, saved image paths.

```
src.data_intake.data_intake.NVRConnection.change_frame_interval(channel: int, frame_interval:  
                                                                int)
```

Update the capture frame interval for a channel and persist it to disk.

Updates the global *camera_frame_intervals* for the given *channel* (stored as a string key) and writes the resulting mapping to FRAME_CONFIG_FILE (*config.json*) as pretty-printed JSON.

Parameters

- **channel** (*int*) – Camera channel identifier to update.
- **frame_interval** (*int*) – New interval expressed in number of frames between saved stills.

Globals modified:

camera_frame_intervals (dict[str, int]): Updated with the new interval for *str(channel)*.

Behaviour:

- Writes the *camera_frame_intervals* mapping to FRAME_CONFIG_FILE.
- Logs success or failure.

Error handling:

- Any exception raised while writing the file is caught and logged; the function does not re-raise exceptions.

Returns

None

async `src.data_intake.data_intake.NVRConnection.channels(request: ChannelsRequest)`

Channel request endpoint will return a set of currently connected channels.

Returns

["1","5","7","12"...] set of camera/channel ids.

Return type

set

Raises

500 Internal Server Error – If channel request fails.

`src.data_intake.data_intake.NVRConnection.getCameraChannels(username: str, password: str, nvr_ip: str)`

Discover available camera channels from the NVR.

Parameters

- **username** (*str*) – NVR username.
- **password** (*str*) – NVR password.
- **nvr_ip** (*str*) – IP address of the NVR.

Returns

A set of detected camera channel ie ["3","7","9"].

Return type

set[str]

Raises

- **socket.error** – If network connection fails.
- **Fault** – If ONVIF SOAP request fails.
- **RequestException** – If HTTP/transport request fails.

async `src.data_intake.data_intake.NVRConnection.health()`

Health check endpoint.

Returns

{"status": "ok"} if service is healthy.

Return type

dict

Raises

500 Internal Server Error – If health check fails.

`src.data_intake.data_intake.NVRConnection.is_reachable(ip, port=80, timeout=2)`

Check if a given host is reachable on a specific port.

Parameters

- **ip** (*str*) – Target IP address.
- **port** (*int, optional*) – Port to test connectivity on. Defaults to 80.
- **timeout** (*int, optional*) – Connection timeout in seconds. Defaults to 2.

Returns

True if the host is reachable, False otherwise.

Return type

bool

```
src.data_intake.data_intake.NVRConnection.onvifCameraInfo(username: str, password: str, nvr_ip: str)
```

Retrieve and log ONVIF camera information from the NVR.

Parameters

- **username** (*str*) – NVR username.
- **password** (*str*) – NVR password.
- **nvr_ip** (*str*) – IP address of the NVR.

Logs:

- Local and UTC system time.
- Camera device time.
- Manufacturer, model, firmware, serial number, and hardware ID.
- ONVIF device service capabilities.

Raises

- **socket.error** – If network connection fails.
- **Fault** – If ONVIF SOAP request fails.
- **RequestException** – If HTTP/transport request fails.

```
async src.data_intake.data_intake.NVRConnection.set_frame_interval(request: FrameIntervalRequest)
```

Update the frame capture interval for a specific camera channel.

Parameters

request ([FrameIntervalRequest](#)) – Channel and interval update request.

Returns

Response with status, updated channel, and new frame interval.

Return type

dict

Raises

500 Internal Server Error – If updating the interval fails.

```
async src.data_intake.data_intake.NVRConnection.startup_event()
```

FastAPI startup handler that initializes camera capture threads and loads config.

This startup event reads NVR credentials from environment variables, retrieves basic ONVIF device information, discovers available camera channels, loads per-channel frame-intervals from `FRAME_CONFIG_FILE` (falls back to `g_frame_interval`), creates manual capture `threading.Event` flags for each channel, and spawns a daemon thread that runs `capture_camera` for each discovered channel.

Globals modified:

`manual_capture_flags` (dict[str, threading.Event]): Mapping of channel -> Event used to trigger manual captures. `camera_frame_intervals` (dict[str, int]): Mapping of channel -> frames-between-captures.

Behavior:

- Calls `onvifCameraInfo` and `getCameraChannels` (network/ONVIF IO).

- May create/modify FRAME_CONFIG_FILE-backed data in memory.
- Starts one daemon thread per channel (threads call *capture_camera*).
- Logs info, warnings, and errors to *logger*.

Raises

ValueError – If NVR_USERNAME or NVR_PASSWORD environment variables are not set.

Returns

None

async `src.data_intake.data_intake.NVRConnection.trigger_capture(payload: CapturePayload)`

src.data_intake.data_intake.utils module

`src.data_intake.data_intake.utils.getMetaDataDir(root_path: str)`

Recursively extract and print EXIF metadata from images in a directory.

Walks through the given directory and all subdirectories, calling *tagReader* on each file found.

Parameters

root_path (*str*) – Path to the root directory containing image files.

Returns

This function prints results to the console.

Return type

None

Raises

FileNotFoundError – If the directory does not exist.

`src.data_intake.data_intake.utils.tagReader(path: str)`

Read and print EXIF metadata from a single image.

Opens the specified image file and extracts EXIF metadata tags, printing each tag and its value to the console.

Parameters

path (*str*) – Path to the image file.

Returns

This function prints results to the console.

Return type

None

Raises

- **FileNotFoundError** – If the image file does not exist.
- **PIL.UnidentifiedImageError** – If the file is not a valid image.

`src.data_intake.data_intake.utils.writeExifTag(image_path: str, author: str = "", serialNumber: str = "", dateTime: str = "", userComment: str = "", description: str = "")`

Write or update EXIF metadata tags in an image.

Updates the EXIF tags for the specified image with provided metadata values such as author, serial number, timestamp, user comments, and description. Existing tags are preserved unless overwritten.

Parameters

- **image_path** (*str*) – Path to the image file.
- **author** (*str*, *optional*) – Camera or photographer name. Defaults to “”.
- **serialNumber** (*str*, *optional*) – Camera serial number. Defaults to “”.
- **dateTime** (*str*, *optional*) – Timestamp of the image in format “YYYY:MM:DD HH:MM:SS”. Defaults to “”.
- **userComment** (*str*, *optional*) – Arbitrary user comment. Defaults to “”.
- **description** (*str*, *optional*) – Description of the image content. Defaults to “”.

Returns

None

Raises

- **FileNotFoundError** – If the image file does not exist.
- **ValueError** – If the provided EXIF fields are invalid.
- **Exception** – For unexpected errors during EXIF writing.

Example

```
>>> writeExifTag(  
...     "photo.jpg",  
...     author="John Doe",  
...     serialNumber="12345XYZ",  
...     dateTime="2025:09:23 10:30:00",  
...     userComment="Inspection photo",  
...     description="Front view of the device"  
... )
```

1.2 src.data_stream package

1.2.1 Submodules

1.2.1.1 src.data_stream.StreamConversionServer module

`src.data_stream.StreamConversionServer.gen_frames(rtspstream)`

Yield frames from an RTSP stream as JPEG-encoded images.

Continuously reads frames from the given RTSP stream, encodes them as JPEG, and yields them in MJPEG-compatible format. If the connection drops, it will automatically attempt to reconnect.

Parameters

rtspstream (*str*) – Full RTSP connection string.

Yields

bytes – A multipart MJPEG frame with headers and encoded JPEG data.

Raises

RuntimeError – If OpenCV cannot encode a frame to JPEG.

`src.data_stream.StreamConversionServer.open_stream(rtsp_url)`

Open and maintain a connection to an RTSP stream.

Attempts to connect to the given RTSP URL using OpenCV’s *VideoCapture*. Retries every 5 seconds until a valid connection is established.

Parameters

rtsp_url (*str*) – Full RTSP connection string.

Returns

An active video capture object connected to the stream.

Return type

cv2.VideoCapture

Raises

- **Exception** – If OpenCV fails to initialize a capture object
- **(unlikely, but possible if OpenCV is misconfigured).** –

`src.data_stream.StreamConversionServer.stream(camera_id)`

Stream MJPEG video from a registered camera.

Provides an MJPEG HTTP endpoint for the specified camera ID. If the camera ID is not registered, returns a 404 error.

Parameters

camera_id (*str*) – The key identifying a registered camera in the *CAMERAS* dictionary.

Returns

Streaming response containing MJPEG frames.

Return type

flask.Response

Raises

werkzeug.exceptions.NotFound – If the camera ID is not registered.

1.3 src.deep_stream package

1.3.1 Submodules

1.3.1.1 src.deep_stream.api module

class `src.deep_stream.api.CameraRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

rtsp_url: `str`

async `src.deep_stream.api.add_source(camera: CameraRequest)`

1.3.1.2 src.deep_stream.deepstream module

`src.deep_stream.deepstream.add_rtsp_source(rtsp_url)`

`src.deep_stream.deepstream.add_sources(data)`

`src.deep_stream.deepstream.bus_call(bus, message, loop)`

`src.deep_stream.deepstream.cb_newpad(decodebin, pad, data)`

`src.deep_stream.deepstream.create_uridecode_bin(index, filename)`

`src.deep_stream.deepstream.decodebin_child_added(child_proxy, Object, name: str, user_data)`

```
src.deep_stream.deepstream.delete_sources(data)

src.deep_stream.deepstream.main(args)

src.deep_stream.deepstream.new_pad_probe(pad, info, user_data)

src.deep_stream.deepstream.pad_debug_probe_limited(pad, info, user_data)
    Prints pad debug info every N frames.

src.deep_stream.deepstream.pgie_frame_probe(pad, info, user_data)
    Probe that prints a message every frame arriving at the inference element.

src.deep_stream.deepstream.print_pipeline_status(pipeline)
    Prints each element and its pads, and checks linking status.

src.deep_stream.deepstream.stop_release_source(source_id)
```

1.4 src.model_deploy package

1.5 src.model_training package

1.5.1 Submodules

1.5.1.1 src.model_training.trainer module

Pig Sorting YOLO Training API

This module provides FastAPI endpoints and helper functions for training and managing Ultralytics YOLO models.

Key Features: - Train YOLO models using a provided dataset (.yaml or .zip format) with configurable hyperparameters. - Stream training progress in real-time via Server-Sent Events (SSE). - Export trained models to ONNX format and validate on the dataset. - Dynamically handle datasets: unzip archives, locate .yaml configuration files. - GPU support with logging of device and CUDA availability. - Threaded training to prevent blocking the API server. - Utility endpoints for retrieving available datasets and projects.

Endpoints: - /training/trainer: Synchronous training, returns trained model path and validation metrics. - /training/stream: Asynchronous training with real-time progress updates. - /training/datasets: List all available datasets and projects.

Dependencies: - ultralytics, torch, fastapi, logging, asyncio, threading, queue, zipfile, os, json

```
class src.model_training.trainer.Queue(maxsize=0)
```

Bases: object

Create a queue object with a given maximum size.

If maxsize is <= 0, the queue size is infinite.

```
empty()
```

Return True if the queue is empty, False otherwise (not reliable!).

This method is likely to be removed at some point. Use `qsize() == 0` as a direct substitute, but be aware that either approach risks a race condition where a queue can grow before the result of `empty()` or `qsize()` can be used.

To create code that needs to wait for all queued tasks to be completed, the preferred technique is to use the `join()` method.

full()

Return True if the queue is full, False otherwise (not reliable!).

This method is likely to be removed at some point. Use `qsize() >= n` as a direct substitute, but be aware that either approach risks a race condition where a queue can shrink before the result of `full()` or `qsize()` can be used.

get(block=True, timeout=None)

Remove and return an item from the queue.

If optional args 'block' is true and 'timeout' is None (the default), block if necessary until an item is available. If 'timeout' is a non-negative number, it blocks at most 'timeout' seconds and raises the Empty exception if no item was available within that time. Otherwise ('block' is false), return an item if one is immediately available, else raise the Empty exception ('timeout' is ignored in that case).

get_nowait()

Remove and return an item from the queue without blocking.

Only get an item if one is immediately available. Otherwise raise the Empty exception.

join()

Blocks until all items in the Queue have been gotten and processed.

The count of unfinished tasks goes up whenever an item is added to the queue. The count goes down whenever a consumer thread calls `task_done()` to indicate the item was retrieved and all work on it is complete.

When the count of unfinished tasks drops to zero, `join()` unblocks.

put(item, block=True, timeout=None)

Put an item into the queue.

If optional args 'block' is true and 'timeout' is None (the default), block if necessary until a free slot is available. If 'timeout' is a non-negative number, it blocks at most 'timeout' seconds and raises the Full exception if no free slot was available within that time. Otherwise ('block' is false), put an item on the queue if a free slot is immediately available, else raise the Full exception ('timeout' is ignored in that case).

put_nowait(item)

Put an item into the queue without blocking.

Only enqueue the item if a free slot is immediately available. Otherwise raise the Full exception.

qsize()

Return the approximate size of the queue (not reliable!).

task_done()

Indicate that a formerly enqueued task is complete.

Used by Queue consumer threads. For each `get()` used to fetch a task, a subsequent call to `task_done()` tells the queue that the processing on the task is complete.

If a `join()` is currently blocking, it will resume when all items have been processed (meaning that a `task_done()` call was received for every item that had been `put()` into the queue).

Raises a `ValueError` if called more times than there were items placed in the queue.

`src.model_training.trainer.find_yaml_file(directory: str) → str`

Locate the first YAML (.yaml or .yml) file in a directory.

Parameters

directory (*str*) – Directory to search.

Returns

Path to YAML file.

Return type

str

Raises

FileNotFoundError – If no YAML file is found.

`src.model_training.trainer.get_data_sets()`

Retrieve all available datasets and projects.

Returns

Paths to dataset files.

Return type

dict

async `src.model_training.trainer.get_mongo_data(model_path: str, dataset_path: str, project_name: str, run_name: str, epochs: int = 1)`

Endpoint for synchronous YOLO model training.

Parameters

- **model_path** (*str*) – Path to pretrained YOLO model.
- **dataset_path** (*str*) – Path to dataset.
- **project_name** (*str*) – Output project directory.
- **run_name** (*str*) – Training run name.
- **epochs** (*int, optional*) – Number of epochs.

Returns

Trained model path, training results, validation results.

Return type

dict

`src.model_training.trainer.prepare_dataset(dataset: str) → str`

Prepare dataset for training by unzipping and locating YAML file.

Parameters

dataset (*str*) – Path to .yaml, .yml, or .zip dataset.

Returns

Path to dataset YAML file.

Return type

str

async `src.model_training.trainer.stream_training(model_path: str, dataset_path: str, project_name: str = 'default', run_name: str = 'run001', epochs: int = 1)`

Endpoint for asynchronous YOLO training with real-time progress via SSE.

Parameters

- **model_path** (*str*) – Path to pretrained YOLO model.
- **dataset_path** (*str*) – Path to dataset.
- **project_name** (*str, optional*) – Output project directory.

- **run_name** (*str*, *optional*) – Training run name.
- **epochs** (*int*, *optional*) – Number of epochs.

Returns

Progress messages as Server-Sent Events.

Return type

StreamingResponse

```
src.model_training.trainer.train_model(model_path: str, dataset_path: str, project_name: str = "",
                                       run_name: str = "", epochs: int = 50, batch: int = 16, lr0: float =
                                       0.002, lrf: float = 0.01, augment: bool = False)
```

Train a YOLO model synchronously.

Parameters

- **model_path** (*str*) – Path to a pretrained YOLO model (.pt).
- **dataset_path** (*str*) – Path to YOLO-formatted dataset (.yaml).
- **project_name** (*str*, *optional*) – Directory name to save outputs.
- **run_name** (*str*, *optional*) – Name of the training run.
- **epochs** (*int*, *optional*) – Number of training epochs.
- **batch** (*int*, *optional*) – Batch size.
- **lr0** (*float*, *optional*) – Initial learning rate.
- **lrf** (*float*, *optional*) – Final learning rate factor.
- **augment** (*bool*, *optional*) – Apply data augmentation.

Returns

Path to best checkpoint, training results, validation results.

Return type

Tuple[str, dict, dict]

```
src.model_training.trainer.train_model_stream(model_path: str, dataset_path: str, queue: Queue,
                                              project_name: str = "", run_name: str = "", epochs: int =
                                              1)
```

Train a YOLO model asynchronously with real-time progress updates.

Parameters

- **model_path** (*str*) – Path to pretrained YOLO model (.pt).
- **dataset_path** (*str*) – Path to dataset (.yaml or .zip).
- **queue** ([Queue](#)) – Queue to stream progress messages to the caller.
- **project_name** (*str*, *optional*) – Directory to save outputs.
- **run_name** (*str*, *optional*) – Name of the training run.
- **epochs** (*int*, *optional*) – Number of epochs.

Returns

None

Notes

- This function is typically run in a separate thread to prevent blocking.
- Real-time epoch progress is pushed into *queue* via the *on_epoch_end* callback.
- Once training is complete, a final message with status 'done' is pushed into the queue.

`src.model_training.trainer.unzip_file(zip_path: str, extract_to: str = '/tmp/unzipped') → str`

Extract ZIP archive to a directory.

Parameters

- **zip_path** (*str*) – Path to ZIP archive.
- **extract_to** (*str*, *optional*) – Extraction directory.

Returns

Path to extracted folder.

Return type

`str`

1.6 src.pig_sorting_api package

1.6.1 Submodules

1.6.1.1 src.pig_sorting_api.client module

Asynchronous MongoDB client module using Motor.

This module sets up an asynchronous connection to a MongoDB instance using credentials from environment variables and exposes:

- *client*: The AsyncIOMotorClient instance connected to the MongoDB server.
- *db*: The *visionPipe* database.
- *db_result*: The *metaData* collection within the *visionPipe* database.

Environment variables required: - MONGO_INITDB_ROOT_USERNAME -
MONGO_INITDB_ROOT_PASSWORD

Usage:

```
from client_async import db_result
```

```
# Example asynchronous query result = await db_result.find_one({"some_field": "value"})
```

1.6.1.2 src.pig_sorting_api.main module

FastAPI API for meat evaluation and image processing pipeline.

This module provides: - Asynchronous MongoDB integration using Motor. - Endpoints for retrieving metadata and images. - Endpoints for triggering model training and chain predictions. - A MongoDB watcher to automatically process newly inserted images. - Utilities to convert images to base64, save annotated predictions, and log results.

Endpoints include: - /api/v1/mongoData : Fetch all metadata from MongoDB. - /api/v1/images/ : Fetch images between given timestamps. - /api/v1/training/trainer : Trigger model training. - /api/v1/training/stream_trainer : Stream training logs from GPU container. - /chain_predict : Forward chain prediction requests to model-deploy. - /save_prediction : Save annotated prediction results to MongoDB. - /api/v1/collector/capture : Trigger manual capture of images. - /api/v1/collector/frame_interval : Set camera frame interval.

Dependencies: - FastAPI - Motor (Async MongoDB client) - httpx (Async HTTP requests) - PIL, OpenCV, NumPy - pymongo, bson

class `src.pig_sorting_api.main.Any(*args, **kwargs)`

Bases: `object`

Special type indicating an unconstrained type.

- Any is compatible with every type.
- Any assumed to have all methods.
- All values assumed to be instances of Any.

Note that all the above statements are true from the point of view of static type checkers. At runtime, Any should not be used with instance checks.

class `src.pig_sorting_api.main.BytesIO(initial_bytes=b'')`

Bases: `_BufferedIOBase`

Buffered I/O implementation using an in-memory bytes buffer.

close()

Disable all I/O operations.

closed

True if the file is closed.

flush()

Does nothing.

getbuffer()

Get a read-write view over the contents of the BytesIO object.

getvalue()

Retrieve the entire contents of the BytesIO object.

isatty()

Always returns False.

BytesIO objects are not connected to a TTY-like device.

read(size=-1, / (Positional-only parameter separator (PEP 570)))

Read at most size bytes, returned as a bytes object.

If the size argument is negative, read until EOF is reached. Return an empty bytes object at EOF.

read1(size=-1, /)

Read at most size bytes, returned as a bytes object.

If the size argument is negative or omitted, read until EOF is reached. Return an empty bytes object at EOF.

readable()

Returns True if the IO object can be read.

readinto(buffer, /)

Read bytes into buffer.

Returns number of bytes read (0 for EOF), or None if the object is set not to block and has no data to read.

readline(*size=-1, /*)

Next line from the file, as a bytes object.

Retain newline. A non-negative size argument limits the maximum number of bytes to return (an incomplete line may be returned then). Return an empty bytes object at EOF.

readlines(*size=None, /*)

List of bytes objects, each a line from the file.

Call readline() repeatedly and return a list of the lines so read. The optional size argument, if given, is an approximate bound on the total number of bytes in the lines returned.

seek(*pos, whence=0, /*)

Change stream position.

Seek to byte offset pos relative to position indicated by whence:

0 Start of stream (the default). pos should be ≥ 0 ; 1 Current position - pos may be negative; 2 End of stream - pos usually negative.

Returns the new absolute position.

seekable()

Returns True if the IO object can be seeked.

tell()

Current file position, an integer.

truncate(*size=None, /*)

Truncate the file to at most size bytes.

Size defaults to the current file position, as returned by tell(). The current file position is unchanged. Returns the new size.

writable()

Returns True if the IO object can be written.

write(*b, /*)

Write bytes to file.

Return the number of bytes written.

writelines(*lines, /*)

Write lines to the file.

Note that newlines are not added. lines can be any iterable object producing bytes-like objects. This is equivalent to calling write() for each element.

class src.pig_sorting_api.main.CaptureRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

Request model for triggering manual data captures.

channels

List of channel IDs to capture from.

Type

list[int]

comment

Optional global comment applied to all captured channels.

Type

str | None

comments

Optional per-channel comments keyed by channel ID.

Type

dict[int, str] | None

channels: list[int]**comment:** str | None = None**comments:** dict[int, str] | None = None**class** src.pig_sorting_api.main.ChainPredictionRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

Request model for chained model predictions.

model_names

Ordered list of model identifiers to execute as a prediction chain.

Type

List[str]

instances

List of input instances to be passed through the model chain. Each dictionary represents a single instance and must conform to the backend model input schema which expects the List to contain base64 encoded images.

Type

List[Dict[str, str]]

parameters

Optional configuration parameters forwarded to the backend prediction service.

Type

Optional[Dict[str, Any]]

instances: List[Dict[str, str]]**model_names:** List[str]**parameters:** Dict[str, Any] | None = None**class** src.pig_sorting_api.main.ChainPredictionResponse(*args: Any, **kwargs: Any)

Bases: BaseModel

predictions: List[Dict[str, Any]]**class** src.pig_sorting_api.main.ChainResult(*args: Any, **kwargs: Any)

Bases: BaseModel

annotated_image: str | None = None**chain:** List[ModelResult]

```
class src.pig_sorting_api.main.ChannelsRequest(*args: Any, **kwargs: Any)
    Bases: BaseModel
    Request model for retrieving active and connected camera ids.

    username
        The username to log into the NVR.
        Type
        str

    password
        The password to log into the NVR.
        Type
        str

    nvr_ip
        The ip to log into the NVR.
        Type
        str

    nvr_ip: str | None = None
    password: str | None = None
    username: str | None = None

class src.pig_sorting_api.main.FrameIntervalRequest(*args: Any, **kwargs: Any)
    Bases: BaseModel
    Request model for updating camera frame capture intervals.

    channel
        Camera channel identifier.
        Type
        int

    interval
        Number of frames between image captures.
        Type
        int

    channel: int
    interval: int

class src.pig_sorting_api.main.InferenceRequest(*args: Any, **kwargs: Any)
    Bases: BaseModel
    confidence: float = 0.5
    image_base64: str
    model_name: str
    return_annotated: bool = True
```



```

class src.pig_sorting_api.main.ModelResult(*args: Any, **kwargs: Any)
    Bases: BaseModel

    detection_count: int

    detections: List[Dict[str, Any]]

    model: str

class src.pig_sorting_api.main.SavePredictionRequest(*args: Any, **kwargs: Any)
    Bases: BaseModel

    Request model for saving annotated prediction results.

    annotated_image
        Base64-encoded image containing model annotations (e.g., bounding boxes, labels).

        Type
        str

    detections
        List of detection results such as bounding boxes, class labels, and confidence scores.

        Type
        List[Dict[str, Any]]

    metadata
        Optional additional metadata associated with the prediction.

        Type
        Optional[Dict[str, Any]]

    ImageName
        Optional base name for the saved image file. If not provided, a timestamp-based name is used.

        Type
        Optional[str]

    ImageName: str | None = None

    annotated_image: str

    detections: List[Dict[str, Any]]

    metadata: Dict[str, Any] | None = None

class src.pig_sorting_api.main.SwineDataSave(*args: Any, **kwargs: Any)
    Bases: BaseModel

    Request model for saving data produced from the swine unit.

    pen
        A string containing the identifier for the pen.

        Type
        str

    rfid
        A string containing the rfid of the pig that triggered the capture.

        Type
        str

```

comment

A string holding any comments for this capture.

Type

Optional(str)

depth_images

A list of depth images encoded in base64.

Type

list(str)

depth_image_names

A list of depth image names that correspond with depth_images.

Type

list(str)

thermal_images

A list of thermal images encoded in base64.

Type

list(str)

thermal_image_names

A list of thermal image names that correspond with thermal_images.

Type

list(str)

rgb_images

A list of rgb images encoded in base64.

Type

list(str)

rgb_image_names

A list of rgb image names that correspond with rgb_images.

Type

list(str)

sensor

Various sensor data.

Type

Optional(Dict[str, *Any*])

timestamp

A timestamp for when the capture happened. The timezone is assumed to be utc

Type

datetime

comment: str | None = None

depth_image_names: List[str]

depth_images: List[str]

pen: str

```

rfid: str

rgb_image_names: List[str]

rgb_images: List[str]

sensor: Dict[str, Any] | None

thermal_image_names: List[str]

thermal_images: List[str]

timestamp: str

```

```
class src.pig_sorting_api.main.TimesRequest(*args: Any, **kwargs: Any)
```

```
Bases: BaseModel
```

```
times: List[str]
```

```
async src.pig_sorting_api.main.capture(request: CaptureRequest)
```

Proxy a manual capture request to the data-collector service.

This endpoint forwards a request to the data-collector to trigger one or more manual captures on the specified channels. Optional comments may be included globally or per channel.

Parameters

request ([CaptureRequest](#)) – Capture configuration including channel identifiers and optional comments.

Returns

JSON response returned by the data-collector service.

Return type

dict

Raises

- **HTTPException** – If the data-collector service is unreachable
- **or returns an unexpected error.** –

```
async src.pig_sorting_api.main.channels(request: ChannelsRequest)
```

Retrieve the list of active and connected camera channels from a network video recorder (NVR).

This endpoint proxies a request to the data-intake service, which connects to the specified NVR and returns a list of currently active and connected camera channels. Optional authentication credentials can be provided if required by the NVR.

Parameters

request ([ChannelsRequest](#)) – NVR connection details and optional credentials.

Returns

JSON response from the data-intake service containing camera channel information.

Return type

dict

Raises

HTTPException – If the data-intake service is unreachable or returns an error.

`src.pig_sorting_api.main.convert_objectid_to_str(data)`

Recursively convert MongoDB ObjectId values to strings.

This function traverses nested dictionaries and lists, replacing any instances of `bson.ObjectId` with their string representation. All other data types are returned unchanged.

Parameters

data (*Any*) – Input data that may contain dictionaries, lists, or `ObjectId` values.

Returns

The input data with all `ObjectId` instances converted to strings.

Return type

Any

`class src.pig_sorting_api.main.datetime(year, month, day[, hour[, minute[, second[, microsecond[, tzinfo ]]]]])`

Bases: *date*

The year, month and day arguments are required. tzinfo may be None, or an instance of a tzinfo subclass. The remaining arguments may be ints.

astimezone()

tz -> convert to local time in new timezone tz

classmethod combine()

date, time -> datetime with same date and time fields

ctime()

Return ctime() style string.

date()

Return date object with same year, month and day.

dst()

Return self.tzinfo.dst(self).

fold

classmethod fromisoformat()

string -> datetime from a string in most ISO 8601 formats

classmethod fromtimestamp()

timestamp[, tz] -> tz's local time from POSIX timestamp.

hour

isoformat()

[sep] -> string in ISO 8601 format, YYYY-MM-DDT[HH[:MM[:SS[.mmm[uuu]]]]][+HH:MM]. sep is used to separate the year from the time, and defaults to 'T'. The optional argument timespec specifies the number of additional terms of the time to include. Valid options are 'auto', 'hours', 'minutes', 'seconds', 'milliseconds' and 'microseconds'.

`max = datetime.datetime(9999, 12, 31, 23, 59, 59, 999999)`

microsecond

`min = datetime.datetime(1, 1, 1, 0, 0)`

minute

classmethod now(*tz=None*)

Returns new datetime object representing current time local to tz.

tz

Timezone object.

If no tz is specified, uses local timezone.

replace()

Return datetime with new specified fields.

resolution = datetime.timedelta(microseconds=1)

second

classmethod strptime()

string, format -> new datetime parsed from a string (like time.strptime()).

time()

Return time object with same time but with tzinfo=None.

timestamp()

Return POSIX timestamp as float.

timetuple()

Return time tuple, compatible with time.localtime().

timetz()

Return time object with same time and tzinfo.

tzinfo

tzname()

Return self.tzinfo.tzname(self).

classmethod utcfromtimestamp()

Construct a naive UTC datetime from a POSIX timestamp.

classmethod utcnow()

Return a new datetime representing UTC day and time.

utcoffset()

Return self.tzinfo.utcoffset(self).

utctimetuple()

Return UTC time tuple, compatible with time.localtime().

async src.pig_sorting_api.main.forward_chain_predict(*request: ChainPredictionRequest*)

Forward a chained prediction request to the model-deploy service.

This endpoint acts as a middleware layer that validates and forwards a chained prediction request to the backend model-deploy service, then returns the validated response to the client.

Parameters

request ([ChainPredictionRequest](#)) – Chained prediction request containing model sequence, input instances, and optional parameters.

Returns

The prediction results returned by the backend service after validation.

Return type*ChainPredictionResponse***Raises****HTTPException –**

- 502 Bad Gateway: If the backend model-deploy service returns an error response. - 500
- Internal Server Error: If an unexpected middleware error occurs.

async `src.pig_sorting_api.main.get_images_by_date(request: TimesRequest) → list[dict]`

Retrieve all images captured within a specified time range.

This endpoint accepts a start and end timestamp, queries the `metaData` MongoDB collection for records within that range, loads the corresponding image files from disk, and returns them as base64-encoded strings along with metadata.

Parameters

request (*TimesRequest*) – Request body containing a list of two ISO 8601 formatted timestamps representing the start and end of the desired time range.

Returns

A list of dictionaries, each containing:

- `filename` (str): Name of the image file.
- `image` (str): Base64-encoded image data.
- `timestamp` (str): ISO 8601 formatted capture time.

Return type`list[dict]`**Raises**

- **HTTPException** – If an unexpected error occurs while querying the
- **database, reading image files, or encoding the images.** –

async `src.pig_sorting_api.main.get_last_checkpoint()`

Load the last checkpoint timestamp from MongoDB.

If no checkpoint is found, returns the current time minus the catch-up window.

Returns

The timestamp of the last processed document.

Return type*datetime*

async `src.pig_sorting_api.main.get_mongo_data() → list[dict]`

Retrieve all metadata records from the MongoDB collection.

This endpoint queries the `metaData` collection in MongoDB, converts the result cursor into JSON-serializable Python objects, and returns all records to the client.

Returns

A list of metadata records retrieved from the `metaData` MongoDB collection.

Return type`list[dict]`**Raises**

- **HTTPException** – If an unexpected error occurs while accessing

- **the database or serializing the results.** –

async `src.pig_sorting_api.main.get_swinedata()`

Retrieve all swinedata records from the MongoDB collection.

This endpoint queries the `swinedata` collection in MongoDB, converts the result cursor into JSON-serializable Python objects, and returns all records to the client.

Returns

A list of swinedata records retrieved from the `swinedata` MongoDB collection.

Return type

`list[dict]`

Raises

- **HTTPException** – If an unexpected error occurs while accessing
- **the database or serializing the results.** –

src.pig_sorting_api.main.image_to_base64(*img: numpy.ndarray*) → *str*

Convert an OpenCV image (NumPy array) to a base64-encoded PNG string.

Parameters

img (*np.ndarray*) – Input image. Can be grayscale or RGB.

Returns

Base64-encoded string of the PNG image.

Return type

str

async `src.pig_sorting_api.main.load_default_models()`

Load default models into the model-deploy service.

This is typically called once at startup to ensure models are available for predictions.

Returns

`None`

async `src.pig_sorting_api.main.proxy_training_stream`(*model_path: str = fastapi.Query*, *dataset_path: str = fastapi.Query*, *project_name: str = fastapi.Query*, *run_name: str = fastapi.Query*, *epochs: int = fastapi.Query*)

Stream a YOLO training job and log results upon completion.

This endpoint forwards a training request to the GPU-enabled model-training service and streams training progress to the client using Server-Sent Events (SSE). Once training completes, the final results are logged to the MongoDB `training_logs` collection.

The following fields are logged to MongoDB:

- `model_path` (str): Path to the pretrained model used.
- `dataset_path` (str): Location of the training dataset.
- `model_export_path` (str): Path to the newly trained model.
- `project` (str): Name of the project.
- `run` (str): Name of the training run.
- `train_results` (dict | str): Training metrics and results.
- `val_results` (dict | str): Validation metrics and results.

- `date_time` (datetime): Timestamp when training completed.

Parameters

- **`model_path`** (*str*) – Path to the pretrained YOLO model.
- **`dataset_path`** (*str*) – Path to the dataset used for training.
- **`project_name`** (*str*) – Logical name for the training project.
- **`run_name`** (*str*) – Identifier for this specific training run.
- **`epochs`** (*int*) – Number of training epochs to run.

Returns

A Server-Sent Events (SSE) stream emitting JSON-encoded training progress updates and a final completion event.

Return type

StreamingResponse

Raises

- **HTTPException** – If the training service fails, the stream cannot
- **be established, or logging to the database is unsuccessful.** –

```
async src.pig_sorting_api.main.run_training(model_path: str = fastapi.Query, dataset_path: str =  
                                             fastapi.Query, project_name: str = fastapi.Query,  
                                             run_name: str = fastapi.Query)
```

Start a YOLO training job with the specified parameters.

This endpoint forwards a training request to the GPU-enabled model-training service, waits for training to complete, and logs the results to the MongoDB `training_logs` collection.

The following fields are logged to MongoDB:

- `model_path` (str): Path to the pretrained model used.
- `dataset_path` (str): Location of the training dataset.
- `model_export_path` (str): Path to the newly trained model.
- `project` (str): Name of the project.
- `run` (str): Name of the training run.
- `train_results` (str): Training metrics and results.
- `val_results` (str): Validation metrics and results.
- `date_time` (datetime): Timestamp when training completed.

Parameters

- **`model_path`** (*str*) – Path to the pretrained YOLO model.
- **`dataset_path`** (*str*) – Path to the dataset used for training.
- **`project_name`** (*str*) – Logical name for the training project.
- **`run_name`** (*str*) – Identifier for this specific training run.

Returns

A dictionary containing:

- `message` (str): Confirmation that training completed.
- `model_export_path` (str): Path to the trained model.
- `train_results` (str): Training results.
- `val_results` (str): Validation results.

Return type

dict

Raises

- **HTTPException** – If the training service fails, times out,
- **or logging to the database is unsuccessful.** –

async `src.pig_sorting_api.main.save_checkpoint`(*ts*: [datetime](#))

Save the last processed timestamp to MongoDB.

Parameters

ts ([datetime](#)) – Timestamp to save as the last processed checkpoint.

Returns

None

async `src.pig_sorting_api.main.save_prediction`(*request*: [SavePredictionRequest](#))

Persist annotated prediction output to disk and MongoDB.

This endpoint decodes a base64-encoded annotated image, saves it to disk using a timestamped or user-provided name, and stores the associated prediction metadata in the MongoDB `predictions` collection.

Parameters

request ([SavePredictionRequest](#)) – Annotated image data, detection results, and optional metadata.

Returns

A dictionary containing:

- `status` (str): Operation status.
- `id` (str): MongoDB ObjectId of the saved prediction.
- `path` (str): Filesystem path to the saved annotated image.

Return type

dict

Raises

- **HTTPException** – If image decoding, file I/O, or database insertion
- **fails.** –

async `src.pig_sorting_api.main.save_swine_data`(*request*: [SwineDataSave](#))

Persist images and pen sensor readings output to disk and MongoDB.

This endpoint decodes base64-encoded annotated images, saves it to disk using a timestamped or user-provided name, and stores the associated prediction metadata such as sensor readings and predicted weight in the MongoDB `predictions` collection.

Parameters

request ([SwineDataSave](#)) –

- `pen` (str): A string containing the identifier for the pen.

- `rfid (str)`: A string containing the rfid of the pig that triggered the capture.
- `comment (Optional(str))`: A string holding any comments for this capture.
- `depth_images (list(str))`: A list of depth images encoded in base64.
- `depth_image_names (list(str))`: A list of depth image names that correspond with `depth_images`.
- `thermal_images (list(str))`: A list of thermal images encoded in base64.
- `thermal_image_names (list(str))`: A list of thermal image names that correspond with `thermal_images`.
- `rgb_images (list(str))`: A list of rgb images encoded in base64.
- `rgb_image_names (list(str))`: A list of rgb image names that correspond with `rgb_images`.
- `sensor (Dict[str, Any])`: Various sensor data.
- `timestamp (datetime)`: A timestamp for when the capture happened. The timezone is assumed to be utc

Returns

A dictionary containing:

- `status (str)`: Operation status.
- `id (str)`: MongoDB ObjectId of the saved prediction.
- `path (str)`: Filesystem path to the saved annotated image.

Return type

dict

Raises

- **HTTPException** – If image decoding, file I/O, or database insertion
- **fails.** –

async `src.pig_sorting_api.main.send_to_chain_prediction(image_path: str)`

Send an image to the chain prediction endpoint and save the annotated results.

This function: - Reads the image asynchronously. - Encodes it as base64. - Sends it to the model-deploy chain prediction endpoint. - Flattens model detection results and attaches model names. - Saves the annotated image to disk. - Stores prediction metadata in MongoDB.

Parameters

image_path (str) – Absolute path to the input image file.

Returns

None

async `src.pig_sorting_api.main.set_frame_interval(request: FrameIntervalRequest)`

Update the frame capture interval for a specific camera.

This endpoint proxies a request to the data-intake service to configure how frequently a camera captures images, based on a frame interval setting.

Parameters

request (FrameIntervalRequest) – Configuration containing the camera channel and desired frame interval.

Returns

JSON response returned by the data-intake service.

Return type

dict

Raises

- **HTTPException** – If the data-intake service is unreachable
- **or returns an unexpected error.** –

```
src.pig_sorting_api.main.start_watcher_loop()
```

```
class src.pig_sorting_api.main.timedelta
```

Bases: object

Difference between two datetime values.

```
timedelta(days=0, seconds=0, microseconds=0, milliseconds=0, minutes=0, hours=0, weeks=0)
```

All arguments are optional and default to 0. Arguments may be integers or floats, and may be positive or negative.

days

Number of days.

```
max = datetime.timedelta(days=999999999, seconds=86399, microseconds=999999)
```

microseconds

Number of microseconds (≥ 0 and less than 1 second).

```
min = datetime.timedelta(days=-999999999)
```

```
resolution = datetime.timedelta(microseconds=1)
```

seconds

Number of seconds (≥ 0 and less than 1 day).

total_seconds()

Total seconds in the duration.

```
class src.pig_sorting_api.main.timezone
```

Bases: *tzinfo*

Fixed offset from UTC implementation of tzinfo.

dst()

Return None.

fromutc()

datetime in UTC -> datetime in local time.

```
max = datetime.timezone(datetime.timedelta(seconds=86340))
```

```
min = datetime.timezone(datetime.timedelta(days=-1, seconds=60))
```

tzname()

If name is specified when timezone is created, returns the name. Otherwise returns offset as 'UTC(+|-)HH:MM'.

utc = `datetime.timezone.utc`

utcoffset()

Return fixed offset.

async `src.pig_sorting_api.main.watch_mongo()`

Poll MongoDB for new images instead of using change streams.

1.7 src.pig_sorting_streamlit package

1.7.1 Submodules

1.7.1.1 src.pig_sorting_streamlit.app module

class `src.pig_sorting_streamlit.app.BytesIO(initial_bytes=b'')`

Bases: `_BufferedIOBase`

Buffered I/O implementation using an in-memory bytes buffer.

close()

Disable all I/O operations.

closed

True if the file is closed.

flush()

Does nothing.

getbuffer()

Get a read-write view over the contents of the BytesIO object.

getvalue()

Retrieve the entire contents of the BytesIO object.

isatty()

Always returns False.

BytesIO objects are not connected to a TTY-like device.

read(size=-1, /)

Read at most size bytes, returned as a bytes object.

If the size argument is negative, read until EOF is reached. Return an empty bytes object at EOF.

read1(size=-1, /)

Read at most size bytes, returned as a bytes object.

If the size argument is negative or omitted, read until EOF is reached. Return an empty bytes object at EOF.

readable()

Returns True if the IO object can be read.

readinto(buffer, /)

Read bytes into buffer.

Returns number of bytes read (0 for EOF), or None if the object is set not to block and has no data to read.

readline(*size=-1, /*)

Next line from the file, as a bytes object.

Retain newline. A non-negative size argument limits the maximum number of bytes to return (an incomplete line may be returned then). Return an empty bytes object at EOF.

readlines(*size=None, /*)

List of bytes objects, each a line from the file.

Call readline() repeatedly and return a list of the lines so read. The optional size argument, if given, is an approximate bound on the total number of bytes in the lines returned.

seek(*pos, whence=0, /*)

Change stream position.

Seek to byte offset pos relative to position indicated by whence:

0 Start of stream (the default). pos should be ≥ 0 ; 1 Current position - pos may be negative; 2 End of stream - pos usually negative.

Returns the new absolute position.

seekable()

Returns True if the IO object can be seeked.

tell()

Current file position, an integer.

truncate(*size=None, /*)

Truncate the file to at most size bytes.

Size defaults to the current file position, as returned by tell(). The current file position is unchanged. Returns the new size.

writable()

Returns True if the IO object can be written.

write(*b, /*)

Write bytes to file.

Return the number of bytes written.

writelines(*lines, /*)

Write lines to the file.

Note that newlines are not added. lines can be any iterable object producing bytes-like objects. This is equivalent to calling write() for each element.

`src.pig_sorting_streamlit.app.camera_select()`

`src.pig_sorting_streamlit.app.chain_prediction(uploaded_files, models_to_chain, return_annotated, confidence)`

Sends request to perform prediction utilizing model(s) and sets the results in session state “prediction_results”

Noindex

Parameters

- **uploaded_files** (*list[UploadedFile]*) – A list of all the images
- **models_to_chain** (*list[Dict]*) – A list of all the models to perform inference with order matters. starts with first element

- **return_annotated** (*bool*) – A flag to tell us if we should return a image, this image will have annotations
- **confidence** (*float*) – This is the threshold for what classes we consider valid and will return 0.5 = 50%

Returns

None

`src.pig_sorting_streamlit.app.display_camera_stream(camera_ids)`

`src.pig_sorting_streamlit.app.display_prediction()`

Looks in the session state for prediction_results and displays the results with the option to save.

`src.pig_sorting_streamlit.app.get_user_frame_interval()`

`src.pig_sorting_streamlit.app.load_models()`

`src.pig_sorting_streamlit.app.parse_channels(camera_ids)`

`src.pig_sorting_streamlit.app.set_camera_interval(camera_ids, frame_between_pic)`

`src.pig_sorting_streamlit.app.set_inference_variables()`

`src.pig_sorting_streamlit.app.trigger_picture(camera_ids, models_to_chain, return_annotated, confidence)`

PYTHON MODULE INDEX

S

- src, ??
- src.data_intake, 1
- src.data_intake.data_intake, 1
- src.data_intake.data_intake.Client, 1
- src.data_intake.data_intake.NVRConnection, 3
- src.data_intake.data_intake.utils, 9
- src.data_stream, 10
- src.data_stream.StreamConversionServer, 10
- src.deep_stream, 11
- src.deep_stream.api, 11
- src.deep_stream.deepstream, 11
- src.model_deploy, 12
- src.model_training, 12
- src.model_training.trainer, 12
- src.pig_sorting_api, 16
- src.pig_sorting_api.client, 16
- src.pig_sorting_api.main, 16
- src.pig_sorting_streamlit, 32
- src.pig_sorting_streamlit.app, 32

INDEX

A

`add_rtsp_source()` (in module `src.deep_stream.deepstream`), 11

`add_source()` (in module `src.deep_stream.api`), 11

`add_sources()` (in module `src.deep_stream.deepstream`), 11

`annotated_image` (`src.pig_sorting_api.main.ChainResult` attribute), 19

`annotated_image` (`src.pig_sorting_api.main.SavePredictionRequest` attribute), 21

`Any` (class in `src.pig_sorting_api.main`), 17

`arbitrary_types_allowed` (`src.data_intake.data_intake.NVRConnection.ImageMetadata.Config` attribute), 5

`astimezone()` (`src.data_intake.data_intake.Client.datetime` method), 1

`astimezone()` (`src.pig_sorting_api.main.datetime` method), 24

`author` (`src.data_intake.data_intake.NVRConnection.ImageMetadata` attribute), 4, 5

B

`bus_call()` (in module `src.deep_stream.deepstream`), 11

`BytesIO` (class in `src.pig_sorting_api.main`), 17

`BytesIO` (class in `src.pig_sorting_streamlit.app`), 32

C

`camera_select()` (in module `src.pig_sorting_streamlit.app`), 33

`CameraRequest` (class in `src.deep_stream.api`), 11

`capture()` (in module `src.pig_sorting_api.main`), 23

`capture_camera()` (in module `src.data_intake.data_intake.NVRConnection`), 5

`CapturePayload` (class in `src.data_intake.data_intake.NVRConnection`), 3

`CaptureRequest` (class in `src.pig_sorting_api.main`), 18

`cb_newpad()` (in module `src.deep_stream.deepstream`), 11

`chain` (`src.pig_sorting_api.main.ChainResult` attribute), 19

`chain_prediction()` (in module `src.pig_sorting_streamlit.app`), 33

`ChainPredictionRequest` (class in `src.pig_sorting_api.main`), 19

`ChainPredictionResponse` (class in `src.pig_sorting_api.main`), 19

`ChainResult` (class in `src.pig_sorting_api.main`), 19

`change_frame_interval()` (in module `src.data_intake.data_intake.NVRConnection`), 6

`channel` (`src.data_intake.data_intake.NVRConnection.FrameIntervalRequest` attribute), 4

`channel` (`src.pig_sorting_api.main.FrameIntervalRequest` attribute), 20

`channels` (`src.data_intake.data_intake.NVRConnection.CapturePayload` attribute), 3

`channels` (`src.pig_sorting_api.main.CaptureRequest` attribute), 18, 19

`channels()` (in module `src.data_intake.data_intake.NVRConnection`), 6

`channels()` (in module `src.pig_sorting_api.main`), 23

`ChannelsRequest` (class in `src.data_intake.data_intake.NVRConnection`), 3

`ChannelsRequest` (class in `src.pig_sorting_api.main`), 19

`close()` (`src.pig_sorting_api.main.BytesIO` method), 17

`close()` (`src.pig_sorting_streamlit.app.BytesIO` method), 32

`closed` (`src.pig_sorting_api.main.BytesIO` attribute), 17

`closed` (`src.pig_sorting_streamlit.app.BytesIO` attribute), 32

`combine()` (`src.data_intake.data_intake.Client.datetime` class method), 1

`combine()` (`src.pig_sorting_api.main.datetime` class method), 24

`comment` (`src.data_intake.data_intake.NVRConnection.CapturePayload` attribute), 3

`comment` (`src.pig_sorting_api.main.CaptureRequest` attribute), 18, 19

`comment` (`src.pig_sorting_api.main.SwineDataSave` at-

tribute), 21, 22

comments (src.data_intake.data_intake.NVRConnection.CaptureRequest attribute), 3

comments (src.pig_sorting_api.main.CaptureRequest attribute), 19

confidence (src.pig_sorting_api.main.InferenceRequest attribute), 20

convert_objectid_to_str() (in module src.pig_sorting_api.main), 23

create_uridecode_bin() (in module src.deep_stream.deepstream), 11

ctime() (src.data_intake.data_intake.Client.datetime method), 1

ctime() (src.pig_sorting_api.main.datetime method), 24

D

date() (src.data_intake.data_intake.Client.datetime method), 1

date() (src.pig_sorting_api.main.datetime method), 24

date_time (src.data_intake.data_intake.NVRConnection.ImageMetadata attribute), 4, 5

datetime (class in src.data_intake.data_intake.Client), 1

datetime (class in src.pig_sorting_api.main), 24

days (src.pig_sorting_api.main.timedelta attribute), 31

decodebin_child_added() (in module src.deep_stream.deepstream), 11

delete_sources() (in module src.deep_stream.deepstream), 11

depth_image_names (src.pig_sorting_api.main.SwineDataSave attribute), 22

depth_images (src.pig_sorting_api.main.SwineDataSave attribute), 22

description (src.data_intake.data_intake.NVRConnection.ImageMetadata attribute), 4, 5

detection_count (src.pig_sorting_api.main.ModelResult attribute), 21

detections (src.pig_sorting_api.main.ModelResult attribute), 21

detections (src.pig_sorting_api.main.SavePredictionRequest attribute), 21

display_camera_stream() (in module src.pig_sorting_streamlit.app), 34

display_prediction() (in module src.pig_sorting_streamlit.app), 34

dst() (src.data_intake.data_intake.Client.datetime method), 1

dst() (src.pig_sorting_api.main.datetime method), 24

dst() (src.pig_sorting_api.main.timezone method), 31

E

empty() (src.model_training.trainer.Queue method), 12

F

find_yaml_file() (in module src.model_training.trainer), 13

flush() (src.pig_sorting_streamlit.app.BytesIO method), 32

fold (src.data_intake.data_intake.Client.datetime attribute), 1

fold (src.pig_sorting_api.main.datetime attribute), 24

forward_chain_predict() (in module src.pig_sorting_api.main), 25

FrameIntervalRequest (class in src.data_intake.data_intake.NVRConnection), 3

FrameIntervalRequest (class in src.pig_sorting_api.main), 20

fromisoformat() (src.data_intake.data_intake.Client.datetime class method), 1

fromisoformat() (src.pig_sorting_api.main.datetime class method), 24

fromtimestamp() (src.data_intake.data_intake.Client.datetime class method), 1

fromtimestamp() (src.pig_sorting_api.main.datetime class method), 24

fromutc() (src.pig_sorting_api.main.timezone method), 31

full() (src.model_training.trainer.Queue method), 12

G

gen_frames() (in module src.data_stream.StreamConversionServer), 10

get() (src.model_training.trainer.Queue method), 13

get_data_sets() (in module src.model_training.trainer), 14

get_images_by_date() (in module src.pig_sorting_api.main), 26

get_last_checkpoint() (in module src.pig_sorting_api.main), 26

get_mongo_data() (in module src.model_training.trainer), 14

get_mongo_data() (in module src.pig_sorting_api.main), 26

get_nowait() (src.model_training.trainer.Queue method), 13

get_swinedata() (in module src.pig_sorting_api.main), 27

get_user_frame_interval() (in module src.pig_sorting_streamlit.app), 34

getbuffer() (src.pig_sorting_api.main.BytesIO method), 17

getbuffer() (src.pig_sorting_streamlit.app.BytesIO method), 32

getCameraChannels() (in module src.data_intake.data_intake.NVRConnection), 7

[getMetaDataDir\(\)](#) (in module [src.pig_sorting_api.main](#)), 27
[src.data_intake.data_intake.utils](#)), 9
[getValue\(\)](#) ([src.pig_sorting_api.main.BytesIO](#) method), 17
[getValue\(\)](#) ([src.pig_sorting_streamlit.app.BytesIO](#) method), 32

H

[health\(\)](#) (in module [src.data_intake.data_intake.NVRConnection](#)), 7
[hour](#) ([src.data_intake.data_intake.Client.datetime](#) attribute), 1
[hour](#) ([src.pig_sorting_api.main.datetime](#) attribute), 24

I

[image_base64](#) ([src.pig_sorting_api.main.InferenceRequest](#) attribute), 20
[image_path](#) ([src.data_intake.data_intake.NVRConnection.FrameIntervalRequest](#) attribute), 4, 5
[image_to_base64\(\)](#) (in module [src.pig_sorting_api.main](#)), 27
[ImageMetadata](#) (class in [src.data_intake.data_intake.NVRConnection](#)), 4
[ImageMetadata.Config](#) (class in [src.data_intake.data_intake.NVRConnection](#)), 5
[ImageName](#) ([src.pig_sorting_api.main.SavePredictionRequest](#) attribute), 21
[InferenceRequest](#) (class in [src.pig_sorting_api.main](#)), 20
[instances](#) ([src.pig_sorting_api.main.ChainPredictionRequest](#) attribute), 19
[interval](#) ([src.data_intake.data_intake.NVRConnection.FrameIntervalRequest](#) attribute), 4
[interval](#) ([src.pig_sorting_api.main.FrameIntervalRequest](#) attribute), 20
[is_reachable\(\)](#) (in module [src.data_intake.data_intake.NVRConnection](#)), 7
[isatty\(\)](#) ([src.pig_sorting_api.main.BytesIO](#) method), 17
[isatty\(\)](#) ([src.pig_sorting_streamlit.app.BytesIO](#) method), 32
[isoformat\(\)](#) ([src.data_intake.data_intake.Client.datetime](#) method), 1
[isoformat\(\)](#) ([src.pig_sorting_api.main.datetime](#) method), 24

J

[join\(\)](#) ([src.model_training.trainer.Queue](#) method), 13

L

[load_default_models\(\)](#) (in module [src.pig_sorting_api.main](#)), 27
[load_models\(\)](#) (in module [src.pig_sorting_streamlit.app](#)), 34
[Lock\(\)](#) (in module [src.data_intake.data_intake.NVRConnection](#)), 5

M

[main\(\)](#) (in module [src.deep_stream.deepstream](#)), 12
[max](#) ([src.data_intake.data_intake.Client.datetime](#) attribute), 2
[max](#) ([src.pig_sorting_api.main.datetime](#) attribute), 24
[max](#) ([src.pig_sorting_api.main.timedelta](#) attribute), 31
[max](#) ([src.pig_sorting_api.main.timezone](#) attribute), 31
[metadata](#) ([src.pig_sorting_api.main.SavePredictionRequest](#) attribute), 21
[microsecond](#) ([src.data_intake.data_intake.Client.datetime](#) attribute), 2
[microsecond](#) ([src.pig_sorting_api.main.datetime](#) attribute), 24
[microseconds](#) ([src.pig_sorting_api.main.timedelta](#) attribute), 31
[min](#) ([src.data_intake.data_intake.Client.datetime](#) attribute), 2
[min](#) ([src.pig_sorting_api.main.datetime](#) attribute), 24
[min](#) ([src.pig_sorting_api.main.timedelta](#) attribute), 31
[min](#) ([src.pig_sorting_api.main.timezone](#) attribute), 31
[minute](#) ([src.data_intake.data_intake.Client.datetime](#) attribute), 2
[minute](#) ([src.pig_sorting_api.main.datetime](#) attribute), 24
[model](#) ([src.pig_sorting_api.main.ModelResult](#) attribute), 21
[model_name](#) ([src.pig_sorting_api.main.InferenceRequest](#) attribute), 20
[model_names](#) ([src.pig_sorting_api.main.ChainPredictionRequest](#) attribute), 19
[ModelResult](#) (class in [src.pig_sorting_api.main](#)), 20
[module](#)
[src](#), 1
[src.data_intake](#), 1
[src.data_intake.data_intake](#), 1
[src.data_intake.data_intake.Client](#), 1
[src.data_intake.data_intake.NVRConnection](#), 3
[src.data_intake.data_intake.utils](#), 9
[src.data_stream](#), 10
[src.data_stream.StreamConversionServer](#), 10
[src.deep_stream](#), 11
[src.deep_stream.api](#), 11
[src.deep_stream.deepstream](#), 11
[src.model_deploy](#), 12
[src.model_training](#), 12
[src.model_training.trainer](#), 12
[src.pig_sorting_api](#), 16

src.pig_sorting_api.client, 16
 src.pig_sorting_api.main, 16
 src.pig_sorting_streamlit, 32
 src.pig_sorting_streamlit.app, 32

N

new_pad_probe() (in module
 src.deep_stream.deepstream), 12
 now() (src.data_intake.data_intake.Client.datetime class
 method), 2
 now() (src.pig_sorting_api.main.datetime class method),
 25
 nvr_ip (src.data_intake.data_intake.NVRConnection.ChannelsRequest
 attribute), 3
 nvr_ip (src.pig_sorting_api.main.ChannelsRequest at-
 tribute), 20

O

onvifCameraInfo() (in module
 src.data_intake.data_intake.NVRConnection),
 8
 open_stream() (in module
 src.data_stream.StreamConversionServer),
 10

P

pad_debug_probe_limited() (in module
 src.deep_stream.deepstream), 12
 parameters (src.pig_sorting_api.main.ChainPredictionRequest
 attribute), 19
 parse_channels() (in module
 src.pig_sorting_streamlit.app), 34
 password (src.data_intake.data_intake.NVRConnection.ChannelsRequest
 attribute), 3
 password (src.pig_sorting_api.main.ChannelsRequest
 attribute), 20
 pen (src.pig_sorting_api.main.SwineDataSave attribute),
 21, 22
 pgie_frame_probe() (in module
 src.deep_stream.deepstream), 12
 predictions (src.pig_sorting_api.main.ChainPredictionRequest
 attribute), 19
 prepare_dataset() (in module
 src.model_training.trainer), 14
 print_pipeline_status() (in module
 src.deep_stream.deepstream), 12
 proxy_training_stream() (in module
 src.pig_sorting_api.main), 27
 put() (src.model_training.trainer.Queue method), 13
 put_nowait() (src.model_training.trainer.Queue
 method), 13

Q

qsize() (src.model_training.trainer.Queue method), 13

Queue (class in src.model_training.trainer), 12

R

read() (src.pig_sorting_api.main.BytesIO method), 17
 read() (src.pig_sorting_streamlit.app.BytesIO method),
 32
 read1() (src.pig_sorting_api.main.BytesIO method), 17
 read1() (src.pig_sorting_streamlit.app.BytesIO
 method), 32
 readable() (src.pig_sorting_api.main.BytesIO
 method), 17
 readable() (src.pig_sorting_streamlit.app.BytesIO
 method), 32
 readinto() (src.pig_sorting_api.main.BytesIO
 method), 17
 readinto() (src.pig_sorting_streamlit.app.BytesIO
 method), 32
 readline() (src.pig_sorting_api.main.BytesIO
 method), 17
 readline() (src.pig_sorting_streamlit.app.BytesIO
 method), 32
 readlines() (src.pig_sorting_api.main.BytesIO
 method), 18
 readlines() (src.pig_sorting_streamlit.app.BytesIO
 method), 33
 replace() (src.data_intake.data_intake.Client.datetime
 method), 2
 replace() (src.pig_sorting_api.main.datetime method),
 25
 RequestException, 5
 resolution (src.data_intake.data_intake.Client.datetime
 attribute), 2
 resolution (src.pig_sorting_api.main.datetime at-
 tribute), 25
 resolution (src.pig_sorting_api.main.timedelta at-
 tribute), 31
 return_annotated (src.pig_sorting_api.main.InferenceRequest
 attribute), 20
 rfid (src.pig_sorting_api.main.SwineDataSave at-
 tribute), 21, 22
 rgb_image_names (src.pig_sorting_api.main.SwineDataSave
 attribute), 22, 23
 rgb_images (src.pig_sorting_api.main.SwineDataSave
 attribute), 22, 23
 rtsp_url (src.deep_stream.api.CameraRequest at-
 tribute), 11
 run_training() (in module src.pig_sorting_api.main),
 28

S

save_checkpoint() (in module
 src.pig_sorting_api.main), 29
 save_prediction() (in module
 src.pig_sorting_api.main), 29

save_swine_data() (in module *src.pig_sorting_api.main*), 29
 SavePictureData() (in module *src.data_intake.data_intake.NVRConnection*), 5
 SavePredictionRequest (class in *src.pig_sorting_api.main*), 21
 second (*src.data_intake.data_intake.Client.datetime* attribute), 2
 second (*src.pig_sorting_api.main.datetime* attribute), 25
 seconds (*src.pig_sorting_api.main.timedelta* attribute), 31
 seek() (*src.pig_sorting_api.main.BytesIO* method), 18
 seek() (*src.pig_sorting_streamlit.app.BytesIO* method), 33
 seekable() (*src.pig_sorting_api.main.BytesIO* method), 18
 seekable() (*src.pig_sorting_streamlit.app.BytesIO* method), 33
 send_to_chain_prediction() (in module *src.pig_sorting_api.main*), 30
 sensor (*src.pig_sorting_api.main.SwineDataSave* attribute), 22, 23
 serial_number (*src.data_intake.data_intake.NVRConnection* attribute), 4, 5
 set_camera_interval() (in module *src.pig_sorting_streamlit.app*), 34
 set_frame_interval() (in module *src.data_intake.data_intake.NVRConnection*), 8
 set_frame_interval() (in module *src.pig_sorting_api.main*), 30
 set_inference_variables() (in module *src.pig_sorting_streamlit.app*), 34
 src
 module, 1
 src.data_intake
 module, 1
 src.data_intake.data_intake
 module, 1
 src.data_intake.data_intake.Client
 module, 1
 src.data_intake.data_intake.NVRConnection
 module, 3
 src.data_intake.data_intake.utils
 module, 9
 src.data_stream
 module, 10
 src.data_stream.StreamConversionServer
 module, 10
 src.deep_stream
 module, 11
 src.deep_stream.api
 module, 11
 src.deep_stream.deepstream
 module, 11
 src.model_deploy
 module, 12
 src.model_training
 module, 12
 src.model_training.trainer
 module, 12
 src.pig_sorting_api
 module, 16
 src.pig_sorting_api.client
 module, 16
 src.pig_sorting_api.main
 module, 16
 src.pig_sorting_streamlit
 module, 32
 src.pig_sorting_streamlit.app
 module, 32
 start_watcher_loop() (in module *src.pig_sorting_api.main*), 31
 startup_event() (in module *src.data_intake.data_intake.NVRConnection*), 8
 stop_background_source() (in module *src.deep_stream.deepstream*), 12
 stream() (in module *src.data_stream.StreamConversionServer*), 11
 stream_training() (in module *src.model_training.trainer*), 14
 strptime() (*src.data_intake.data_intake.Client.datetime* class method), 2
 strptime() (*src.pig_sorting_api.main.datetime* class method), 25
 SwineDataSave (class in *src.pig_sorting_api.main*), 21
T
 tagReader() (in module *src.data_intake.data_intake.utils*), 9
 task_done() (*src.model_training.trainer.Queue* method), 13
 tell() (*src.pig_sorting_api.main.BytesIO* method), 18
 tell() (*src.pig_sorting_streamlit.app.BytesIO* method), 33
 thermal_image_names
 (*src.pig_sorting_api.main.SwineDataSave* attribute), 22, 23
 thermal_images (*src.pig_sorting_api.main.SwineDataSave* attribute), 22, 23
 time() (*src.data_intake.data_intake.Client.datetime* method), 2
 time() (*src.pig_sorting_api.main.datetime* method), 25
 timedelta (class in *src.pig_sorting_api.main*), 31
 times (*src.pig_sorting_api.main.TimesRequest* attribute), 23

`TimesRequest` (*class in src.pig_sorting_api.main*), 23
`timestamp` (*src.pig_sorting_api.main.SwineDataSave attribute*), 22, 23
`timestamp()` (*src.data_intake.data_intake.Client.datetime method*), 2
`timestamp()` (*src.pig_sorting_api.main.datetime method*), 25
`timetuple()` (*src.data_intake.data_intake.Client.datetime method*), 2
`timetuple()` (*src.pig_sorting_api.main.datetime method*), 25
`timetz()` (*src.data_intake.data_intake.Client.datetime method*), 2
`timetz()` (*src.pig_sorting_api.main.datetime method*), 25
`timezone` (*class in src.pig_sorting_api.main*), 31
`total_seconds()` (*src.pig_sorting_api.main.timedelta method*), 31
`train_model()` (*in module src.model_training.trainer*), 15
`train_model_stream()` (*in module src.model_training.trainer*), 15
`trigger_capture()` (*in module src.data_intake.data_intake.NVRConnection*), 9
`trigger_picture()` (*in module src.pig_sorting_streamlit.app*), 34
`truncate()` (*src.pig_sorting_api.main.BytesIO method*), 18
`truncate()` (*src.pig_sorting_streamlit.app.BytesIO method*), 33
`tzinfo` (*src.data_intake.data_intake.Client.datetime attribute*), 2
`tzinfo` (*src.pig_sorting_api.main.datetime attribute*), 25
`tzname()` (*src.data_intake.data_intake.Client.datetime method*), 2
`tzname()` (*src.pig_sorting_api.main.datetime method*), 25
`tzname()` (*src.pig_sorting_api.main.timezone method*), 31

U

`unzip_file()` (*in module src.model_training.trainer*), 16
`user_comment` (*src.data_intake.data_intake.NVRConnection.ImageMetadata attribute*), 4, 5
`username` (*src.data_intake.data_intake.NVRConnection.ChannelsRequest attribute*), 3
`username` (*src.pig_sorting_api.main.ChannelsRequest attribute*), 20
`utc` (*src.pig_sorting_api.main.timezone attribute*), 31
`utcfromtimestamp()` (*src.data_intake.data_intake.Client.datetime class method*), 2
`utcfromtimestamp()` (*src.pig_sorting_api.main.datetime class method*), 25
`utcnow()` (*src.data_intake.data_intake.Client.datetime class method*), 2
`utcnow()` (*src.pig_sorting_api.main.datetime class method*), 25
`utcoffset()` (*src.data_intake.data_intake.Client.datetime method*), 2
`utcoffset()` (*src.pig_sorting_api.main.datetime method*), 25
`utcoffset()` (*src.pig_sorting_api.main.timezone method*), 32
`utctimetuple()` (*src.data_intake.data_intake.Client.datetime method*), 2
`utctimetuple()` (*src.pig_sorting_api.main.datetime method*), 25

W

`watch_mongo()` (*in module src.pig_sorting_api.main*), 32
`writable()` (*src.pig_sorting_api.main.BytesIO method*), 18
`writable()` (*src.pig_sorting_streamlit.app.BytesIO method*), 33
`write()` (*src.pig_sorting_api.main.BytesIO method*), 18
`write()` (*src.pig_sorting_streamlit.app.BytesIO method*), 33
`writeExifTag()` (*in module src.data_intake.data_intake.utils*), 9
`writelines()` (*src.pig_sorting_api.main.BytesIO method*), 18
`writelines()` (*src.pig_sorting_streamlit.app.BytesIO method*), 33